Unit 4

Exception Handling and Multithreading

(Marks : 12)

Errors and Exceptions

Error

Errors are the wrongs that can make a program to go wrong. An error may produce an incorrect output or may terminate the execution of the program abruptly or even may cause the system to crash.

Types of error

- Compile Time Error
- Run Time Error

Compile Time Error

- All syntax errors are detected and displayed by the Java compiler, therefore these errors are known as compile time errors; which generates a message indicating the type of error and the position in the Java source file where the error occurred.

- The most common compile time errors are:
  - Missing semicolon
  - Missing brackets in classes and methods
  - Misspelling of identifiers and keywords
  - Missing double quotes
  - Use of undeclared variables
  - And so on…

Runtime Error

- An error that occurs during the execution of a program are called Run Time Error. That is , a program may compile successfully creating the .class file but may not run properly. Such programs may produce wrong result due to wrong logic or may terminate due to error such as stack overflow. Runtime errors indicate bugs in the program.

- The most common runtime errors are:
  - Dividing an integer by zero
  - Accessing an element that is out of the bounds of an array
  - Trying to store a value into an array of an incompatible class or type
  - Accessing a character that is out of bounds of a string
  - And do on…

Exception

- An exception is a condition that is caused by a run-time error in the program.
- A Java Exception is an object that describes the exception that occurs in a program.
- Exception is class, present in java.lang.*; package
- When an exceptional events occurs in java, an exception is said to be thrown. The code that is responsible for doing something about the exception is called an **exception handler**.

Exception

- **It is a condition caused (occur) due to Run time error in program.**
- **Exception is class, present in java.lang.*; package**
- **It use to handle special errors comes in program.**
- **If any Exception occur during execution of program the *object* of Exception class will throw the error message to user, for *taking certain action* against the error.**

Types of Exception

- **Checked Exception**

  The classes that extend Throwable class except RuntimeException and Error are known as checked exceptions.

  e.g. *IOException, SQLException* etc.

  Checked exceptions are checked at compile-time.

- **Unchecked Exception**

  The classes that extend RuntimeException are known as unchecked exceptions.

  e.g. *ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException* etc.

  Unchecked exceptions are not checked at compile-time rather they are checked at runtime. Unchecked exception are ignored at compile time.

```
class UncaughtException
{
    public static void main(String args[])
    {
        int a = 0;
         int b = 7/a; // Divide by zero, will lead to exception
    }
}
```

name and description of Exception

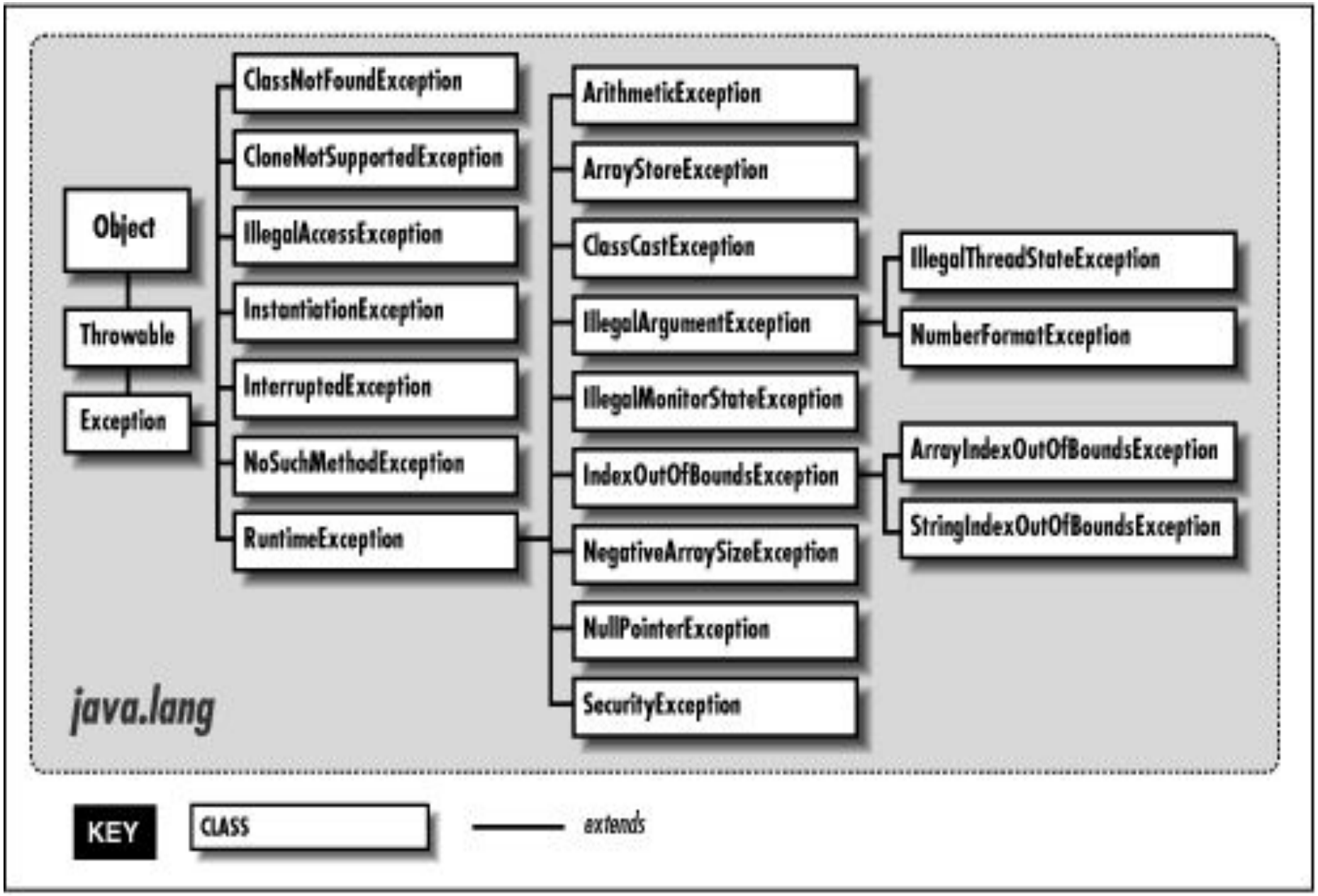
```
java.lang.ArithmeticException: / by zero
    at UncaughtException.main(UncaughtException.java:4)
```

class name

file name

Stack Trace
(line at which exception occured)

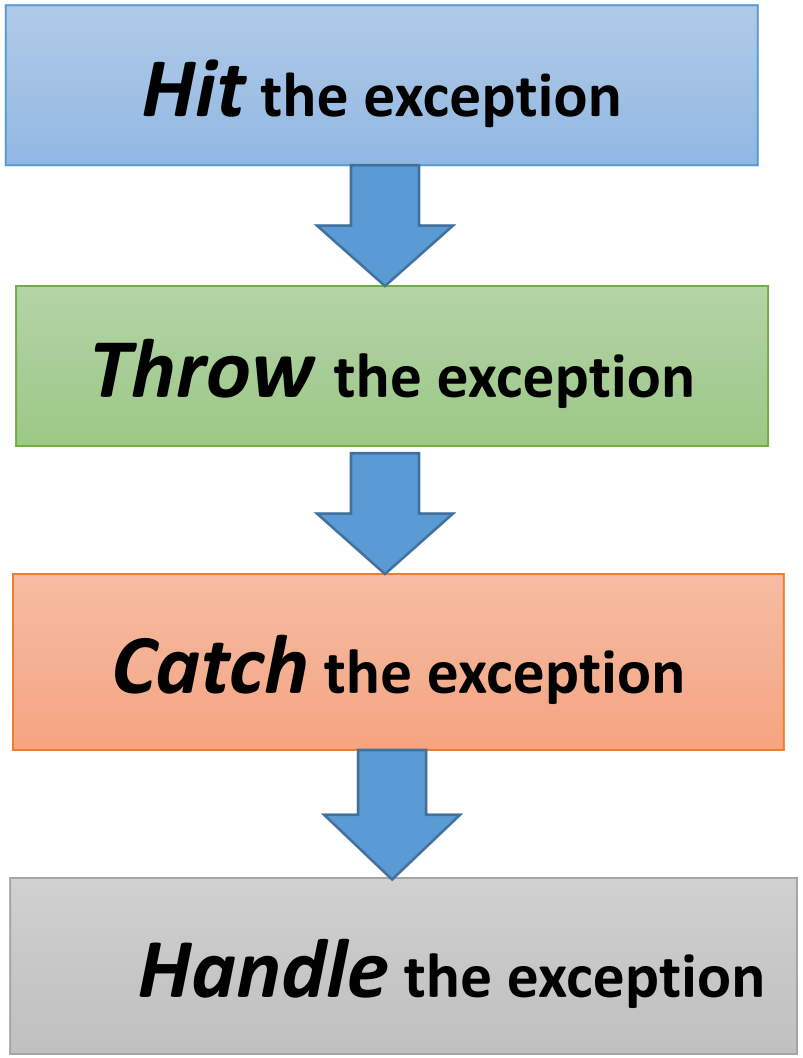
Common Exception : Classification



Un-caught Exception

- Whenever Exception occur in program it must handle by Exception Handler.
- In JAVA by default the 'default exception handler' is present
- It display name of exception and where it occurred
- It terminate program and not execute any statement after that.

Exception Handling process



try block

- A block of code which checks the statements for an error or exception.
- Java try block must be followed by either catch or finally block.
- Syntax of java try

```
try
    {
     //code that may throw exception
     }
```

catch block

- Java catch block is used to handle the Exception. It must be used after the try block only.
- A try and catch always form in couple.
- You can use multiple catch block with a single try.

```
catch (Exception_class object_name)
{
    //statements…
}
```

try-catch block

```
try {
    stmts
     }
catch(Exception e)
  {
   ......
  }
```

Problem without exception handling

```
public class Test
{
  public static void main(String args[])
  {
      int data=50/0;       //may throw exception
      System.out.println("rest of the code...");
  }
}
```

Output:

Exception in thread main java.lang.ArithmeticException:/ by zero

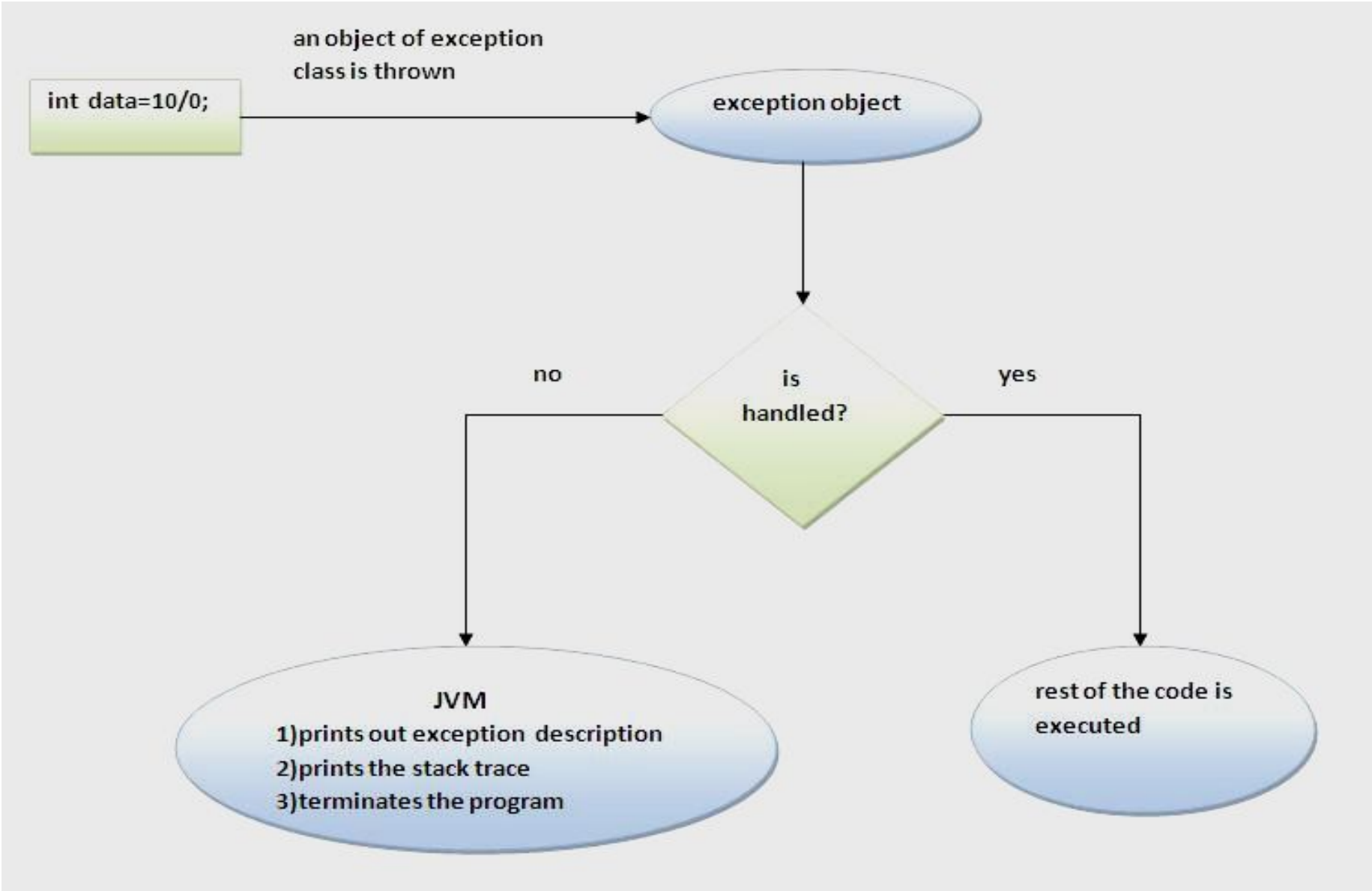
Solution by exception handling

```
public class Test
 {
  public static void main(String args[])   {
     try
     {
            int data=50/0;
         }
     catch(ArithmeticException e)
     {
      System.out.println(e);
     }
        System.out.println("rest of the code...");
  }
}
```

Output:

Exception in thread main java.lang.ArithmeticException:/ by zero
rest of the code...

Working of java try-catch block



Multiple catch blocks

- A try block can be followed by multiple catch blocks. You can have any number of catch blocks after a single try block.
- If an exception occurs in the try block, the exception is passed to the first catch block in the list.
- If the exception type of exception, matches with the first catch block it gets caught, if not the exception is passed down to the next catch block. This continue until the exception is caught or falls through all catches.

Example for Multiple Catch blocks

```
class Excep
{
   public static void main(String[] args)
   {
      try
      {
                        int n = Integer.parseInt(args[0]);
            int arr[]={1,2};
            arr[2]=3/n;
      }
      catch(ArithmeticException ae)
      {
            System.out.println("divide by zero");
      }
      catch(ArrayIndexOutOfBoundsException e)
      {
            System.out.println("array index out of bound exception");
      }
   }
}
```

Output :

divide by zero

Nested try statement

- try statement can be nested inside another block of try.
- Nested try block is used when a part of a block may cause one error while entire block may cause another error.
- In case if inner try block does not have a catch handler for a particular exception then the outer try is checked for match.

Syntax:

```
....
try
{
    statement 1;
    statement 2;
    try
    {
        statement 1;
        statement 2;
    }
    catch(Exception e)
    {
    }
}
catch(Exception e)
{
}
....
```

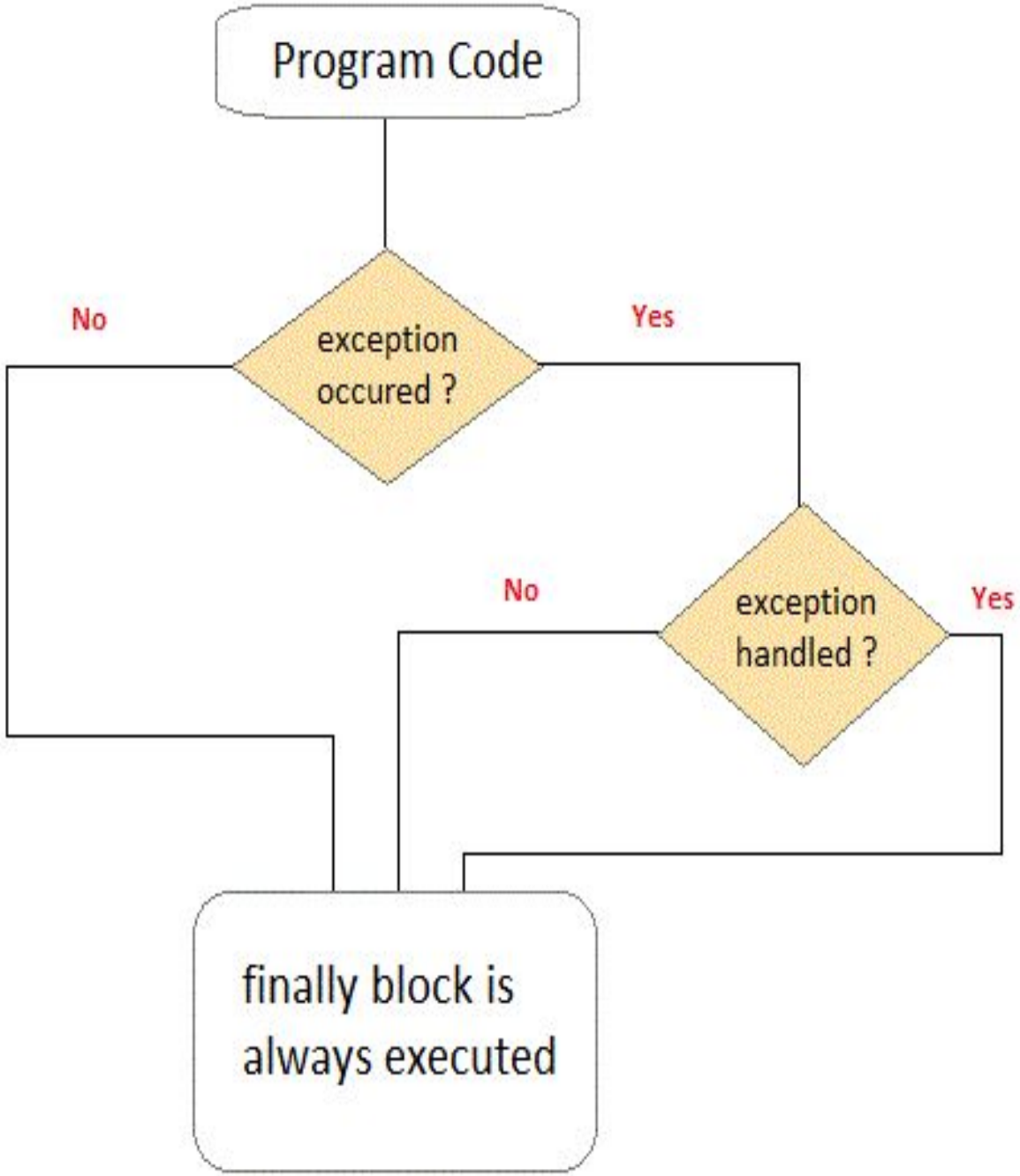
Example of Nested try statement

```
class Excep
{
   public static void main(String[] args)
   {
      try
      {
            int arr[]={5,0,1,2};
            try
            {
                  int x=arr[3]/arr[1];
            }
            catch(ArithmeticException ae)
            {
                  System.out.println("divide by zero");
            }
            arr[4]=3;
      }
      catch(ArrayIndexOutOfBoundsException e)
      {
            System.out.println("array index out of bound exception");
      }
   }
}
```

finally clause

- **Java finally block** is a block that is used *to execute important code* such as closing connection, stream etc.
- Java finally block is always executed whether exception is handled or not.
- Java finally block follows try or catch block.

```
finally
{
    //statements
}
```


Program Code
exception occured ?
No
Yes
exception handled ?
No
Yes
finally block is always executed

Example demonstrating finally Clause

```
Class ExceptionTest
{
   public static void main(String[] args)
   {
      int a[]= new int[2];
      System.out.println("out of try");
      try
      {
            System.out.println("Access invalid element"+ a[3]);
            /* the above statement will throw ArrayIndexOutOfBoundException */
      }
      finally
      {
            System.out.println("finally is always executed.");
      }
   }
}
```

Output :

Out of try
finally is always executed.
Exception in thread main java. Lang. exception array Index out of bound exception.

throw Keyword

- If we want to throw the exceptions (System / user defined) by our own, the throw statement can be used.
- Java provides a keyword “throw” using which we can explicitly throw the exceptions in the code
- The throw keyword in Java is used to throw an exception from a method or block of code when an error or exceptional condition occurs that the program cannot handle at runtime.

- Syntax:

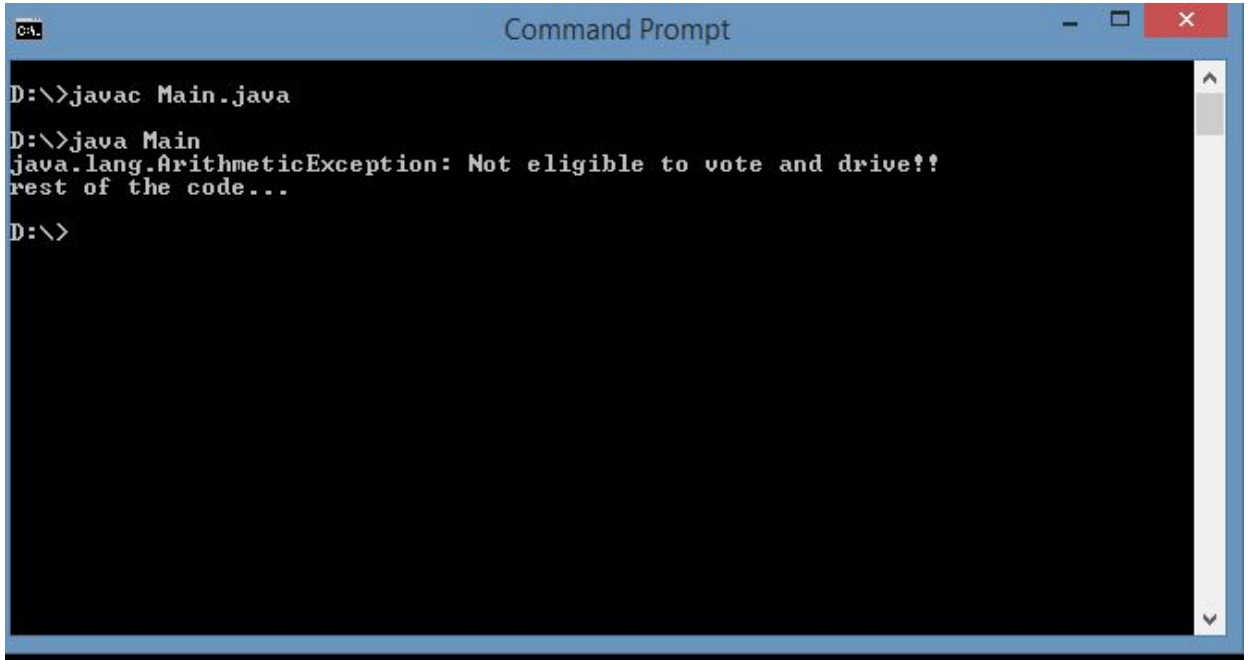
```
throw ThrowableInstance;
```

or

```
throw new exception_class("error message");
```

Example of throw Keyword

```
class Main    {
   static void validate_Age(int age)
   {
    try
     {
      if(age<18)
                throw new ArithmeticException("Not eligible to vote and drive!!");
     }
     catch(ArithmeticException e)
     {
      System.out.println(e);
     }
   }
   public static void main(String args[])   {
      validate_Age(10);
      System.out.println("rest of the code...");
  }
}
```



throws Keyword

- To handle System Defined Exception (at compile time only) without creating the object.
- throws is a keyword in Java that is used in the signature of a method to indicate that this method might throw one of the listed type exceptions. The caller to these methods has to handle the exception using a try-catch block.
- With the help of the throws keyword, we can provide information to the caller of the method about the exception.
- **Syntax :**

type method_name(parameter_list) **throws** *exception_list*

```
{
  //definition of method
}
```

Example demonstrating throws Keyword

```
class Test
{
  public static void main(String args[]) throws ArithmeticException
  {
        DataInputStream din=new DataInputStream(System.in);
            int no=Integer.parseInt(din.readLine());
        int a=no/0;
            System.out.println(a);
  }
}
```

Creating User defined Exception

- Create a custom or user exception class that extends the base Exception class (java.lang.Exception).
- Define the constructor for the custom or user exception class. The constructor can accept string parameter to provide more information about the error.
- Override the toString() method to provide a custom error message.
- Use the "throw" keyword to throw an instance of the custom or use exception when the error condition occurs.
- Use a "try-catch" block to catch the exception and handle it appropriately.

Creating our own Exception

```
import java.lang.Exception;
import java.io.*;

class MyException extends Exception
{
	MyException(String msg)
	{
		super(msg);
	}
}
```

```
class exp
{
	public static void main(String args[])
	{
	try
	{
		Scanner br=new Scanner(System.in);
		System.out.println("Enter Number:");
		int n=br.nextInt();
		if(n%2!=0)
		{
		  throw new MyException("Number is Not Even Number");
	} catch(MyException e)
	{
		System.out.println(e.getMessage());
	}
	}
}
```

Multithreading

Introduction

- The process of executing multiple threads simultaneously is known as multithreading. i.e. Main program (Process) is divided into two or more subprogram (Processes) which can be executing at the same time in parallel.

- The main purpose of multithreading is to maximize the CPU time utilization.

- A multithreaded program contains two or more parts that can run concurrently. Each such part of a program is called thread.

Thread

- It is a sub program (small program) that has a single flow of control.
- Multiple threads share a common memory space.
- Also called as Light Weight Process that can run concurrently with the other parts(other threads) of the same process.
- Thread is similar to a program that has single flow of control.
- Threads are **independent** so it doesn't affect other threads if exception occur in a single thread.

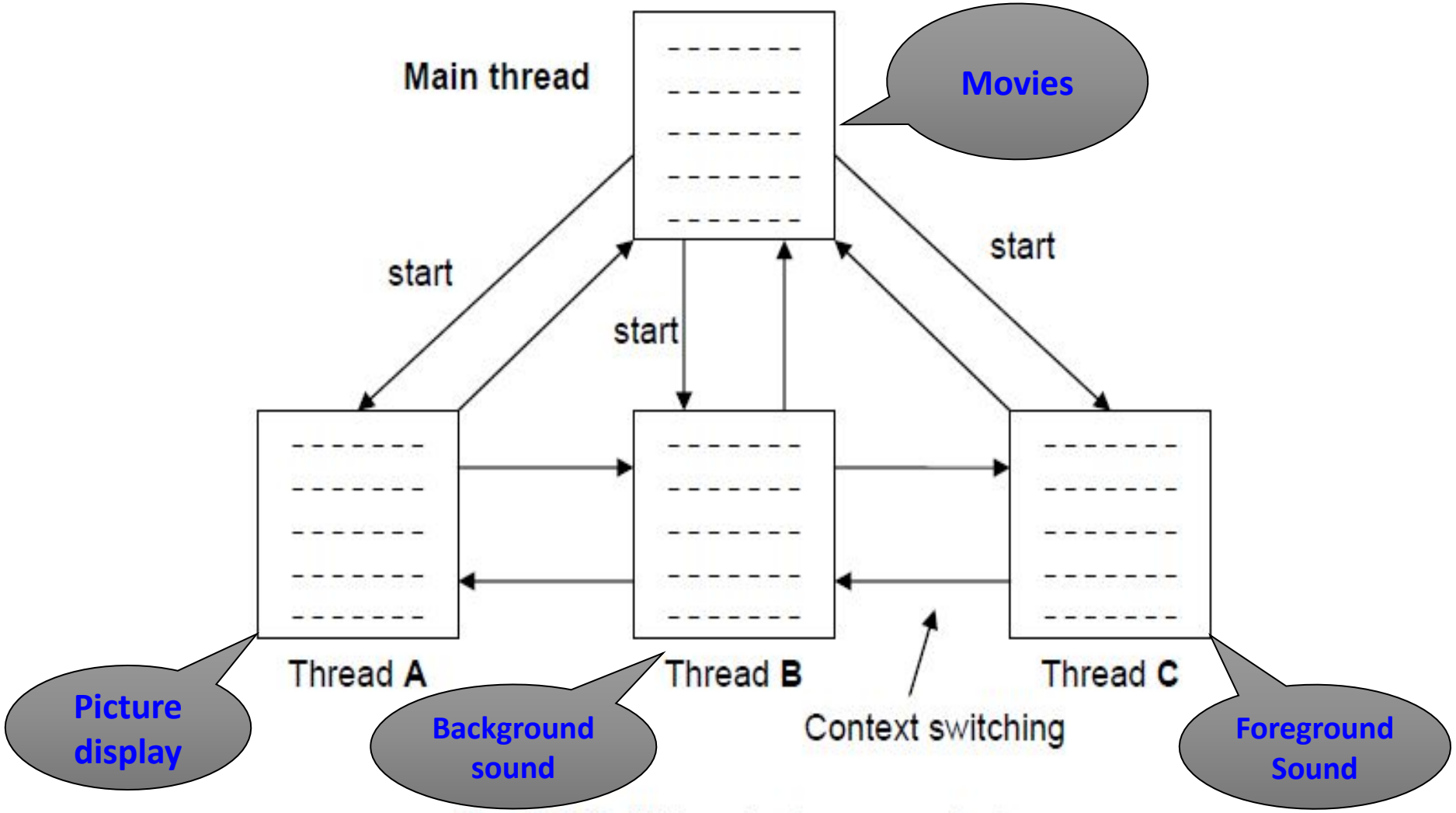


Fig. 5.3 Multithreaded program in Java

Creating Threads

Two ways to create thread in JAVA

1. extending Thread class using extends keyword

2. Implementing Runnable interface

1. Extending Thread class

- Create a class that extends the java.lang.Thread class.

```
class classname extends Thread
{    }
```

- This class overrides the run() method available in the Thread class. A thread begins its life inside run() method.

```
public void run()
{  }
```

- We create an object of our new class and call start() method to start the execution of a thread.

```
classname obj = new classname();
obj.start();
```

- Start() invokes the run() method on the Thread object.

Example

```
class MyThread extends Thread
{
   public void run()
   {
      System.out.println("Thread started running..");
   }
}
classMyThreadDemo
{
   public static void main( String args[] )
   {
      MyThread mt = new MyThread();
      mt.start();
```

Output :
Thread started running..

2. Implementing Runnable interface

- We create a new class which implements java.lang.Runnable interface

```
class classname implements Runnable
{   }
```

- Override run() method inside this class.

```
public void run()
{   }
```

- Then we instantiate a Thread object and call start() method on this object.

```
classname obj=new classname();
Thread t=new Thread(obj);
t.start();
```

Example

```
class MyThread implements Runnable
{
   public void run()
   {
      System.out.println("Thread started running..");
   }
}
class MyThreadDemo
{
   public static void main( String args[] )
   {
      MyThread mt = new MyThread();
      Thread t = new Thread(mt);
      t.start();
```

Output :
Thread started running..

```
class X implements Runnable
{
   public void run()
   {
     for(int i=1;i<=5;i++)
       System.out.println("\tThread X  "+i);
     }
}
class Y implements Runnable
{
public void run()
   {
    for(int j=5;j>=1;j--)
     System.out.println("\t\tThread Y  "+j);
   }     }
```

```
class RunnableTest
{
    public static void main(String args[])
   {
        X objX = new X();
        Thread threadX = new Thread(objX);
        Y objY = new Y();
        Thread threadY = new Thread(objY);
        threadX.start();
        threadY.start();
       }
  }
```

- **Output:**

```
C:\Program Files\java\jdk1.7.0\bin>java RunnableTest
      Thread X  1
            Thread Y  5
      Thread X  2
            Thread Y  4
      Thread X  3
            Thread Y  3
      Thread X  4
            Thread Y  2
      Thread X  5
            Thread Y  1
```

Life Cycle of a Thread

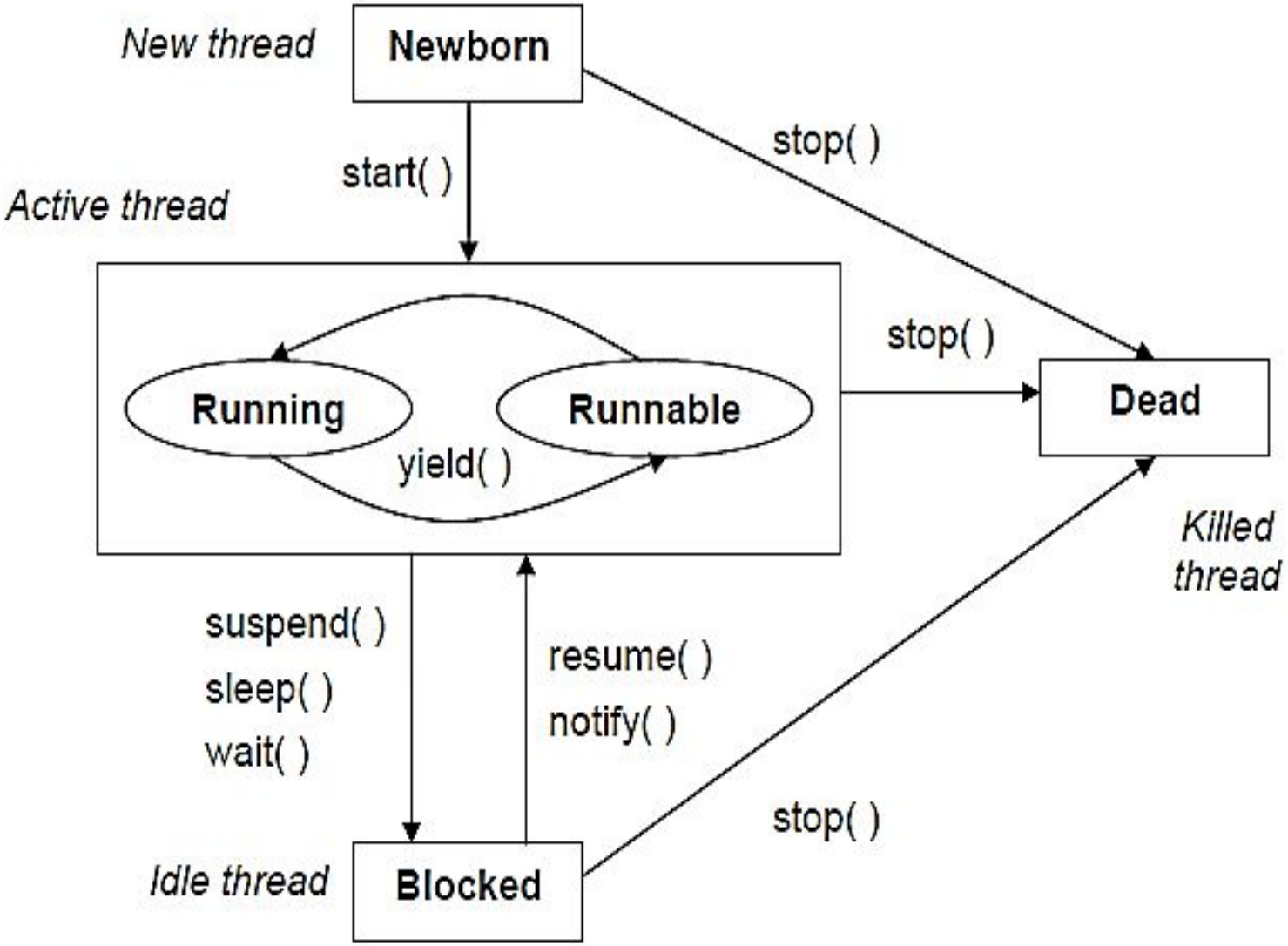


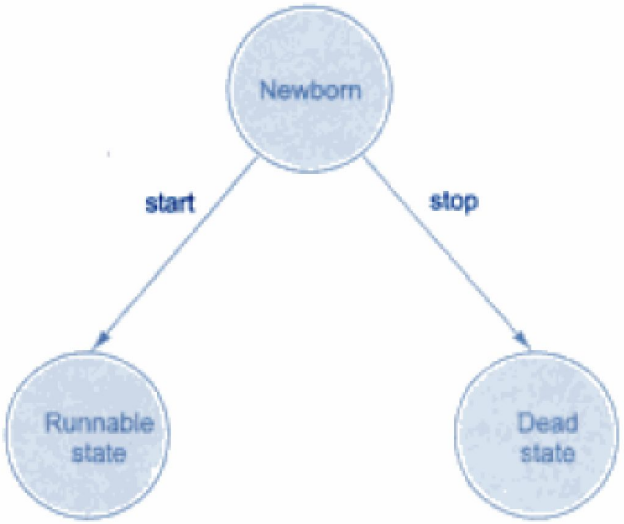
Fig. 5.4 Thread life-cycle

Life cycle of thread

1. **Newborn thread:** creating new thread
2. **Runnable thread:** ready for execution
3. **Running thread:** being executing thread
4. **Blocked thread:** temporary Suspend thread
5. **Dead (kill) thread:** end of life of thread

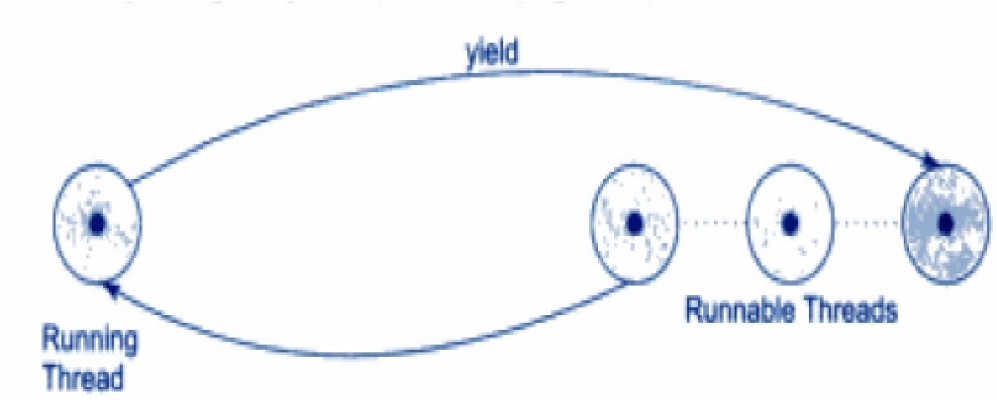
1. Newborn state:

- When a thread object is created it is said to be in a new born state.
- When the thread is in a new born state it is not scheduled running from this state.
- It can be scheduled for running by start() method or killed by stop() method.
- If it is put in a queue it moves to runnable state.



2. **Runnable State:**

- It means that thread is ready for execution and is waiting in queue for the availability of the processor.
- If all threads have equal priority then they are given time slots for execution in round robin fashion.
- The thread that relinquishes control joins the queue at the end and again waits for its turn.
- A thread can relinquish the control to another thread before its turn comes by yield().



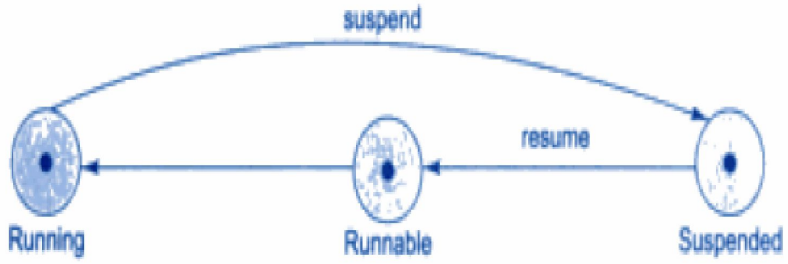
3. **Running State:**

- It means that the processor has given its time to the thread for execution.
- The thread runs until it relinquishes control on its own or it is pre-empted by a higher priority thread.

4. Blocked state:

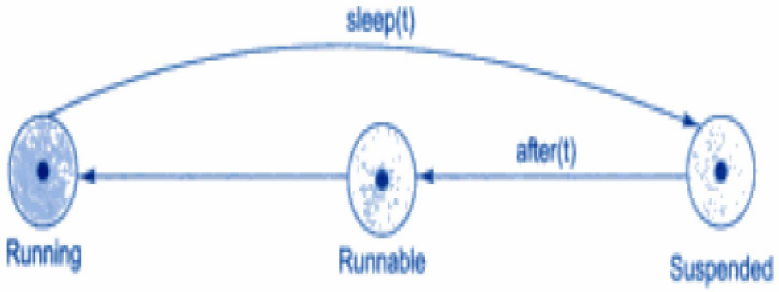
A thread can be temporarily suspended or blocked from entering into the runnable and running state by using either of the following thread method.

- ***Suspend()*** : Thread can be suspended by this method. It can be rescheduled by resume().



- ***Wait()*** : If a thread requires to wait until some event occurs, it can be done using wait() method and can be scheduled to run again by notify().

- ***Sleep()*** **:** We can put a thread to sleep for a specified time period using *sleep(time)* where time is in milliseconds. It re-enters the runnable state as soon as period has elapsed /over



5. Dead State:

- Whenever we want to stop a thread from running further we can call its stop().
- This method causes the thread to move to a dead state.
- A thread will also move to dead state automatically when it reaches to end of the method.
- The stop() method may be used when the premature death is required.

Methods of Thread class…

Method	Meaning
void setName(String str)	Set the name for Thread
String getName()	Obtain the thread's name
void setPriority(int i)	Set the manual priority to thread
int getPriority()	Obtain the thread's priority
boolean isAlive()	Determines whether thread is still running?
void run()	Create newborn thread
void start()	To start the execution of newborn thread
void sleep(int time)	Blocked thread in specific amount of time
void suspend()	Blocked until further order
void wait()	Blocked until certain condition occur
void stop()	Kill the thread forcefully…
void notify()	Get the notification of first thread waiting in queue
void NotifyAll()	Get the notification of all the thread waiting in queue.

Sample code on ***sleep*** method

```
class threadB extends Thread
{
        public void run()
        {
                for(int i=1;i<=5;i++)
                {
                        System.out.println("threa-B: "+i);
                        try{
                                Thread.sleep(1000);
                            }
                        catch(Exception e)
                        {
                        }
                }
        }
}
class multithreading
{
        public static void main(String asd[]) throws Exception
        {

                threadB B=new threadB();
                B.start();
        }
}
```

Thread Priority

- Each thread have a priority.
- Priorities are represented by a number between 1 and 10.
- In most cases, thread schedular schedules the threads according to their priority (known as pre-emptive scheduling).
- But it is not guaranteed because it depends on JVM specification that which scheduling it chooses.

- 3 constants defined in Thread class:
  - public static int **MIN_PRIORITY**
  - public static int **NORM_PRIORITY**
  - public static int **MAX_PRIORITY**

- The value of Default priority i.e. **NORM_PRIORITY is 5**.
- The value of **MIN_PRIORITY is 1** and the value of **MAX_PRIORITY is 10**.

Thread Priority Methods

- To set a thread's priority, use the **setPriority()** method, which is a member of **Thread**. This is its general form:

  void setPriority(int *level*)

- You can obtain the current priority setting by calling the **getPriority( )** method of **Thread**, shown here:

  int getPriority( )

Example of priority of a Thread

```
class TestMultiPriority1 extends Thread
{
  public void run()
  {
      System.out.println("running thread name is:” + Thread.currentThread().getName());
      System.out.println("running thread priority is:” + Thread.currentThread().getPriority());
  }
  public static void main(String args[]) {
        TestMultiPriority1 m1=new TestMultiPriority1();
        TestMultiPriority1 m2=new TestMultiPriority1();
        m1.setPriority(Thread.MIN_PRIORITY);
        m2.setPriority(Thread.MAX_PRIORITY);
        m1.start();
        m2.start();
      }
}
```

Synchronization

- At times when more than one thread try to access a shared resource, we need to ensure that resource will be used by only one thread at a time.
- The process by which this is achieved is called **synchronization**.
- The synchronized keyword in java creates a block of code referred to as critical section.

- **General Syntax :**

Synchronized Method:

```
<access modifier> synchronized method_name (parameter)
{
//code
}
```

Synchronized object:

```
synchronized (object)
  {
    //statement to be synchronized
  }
```

Every Java object with a critical section of code gets a lock associated with the object. To enter critical section a thread need to obtain the corresponding object's lock.

Need of Synchronization

- If we do not use synchronization, and let two or more threads access a shared resource at the same time, it will lead to distorted results.
- For example, Suppose we have two different threads T1 and T2, T1 starts execution and save certain values in a file *temporary.txt* which will be used to calculate some result when T1 returns. Meanwhile, T2 starts and before T1 returns, T2 change the values saved by T1 in the file *temporary.txt* (*temporary.txt* is the shared resource). Now obviously T1 will return wrong result.
- To prevent such problems, synchronization was introduced. With synchronization in above case, once T1 starts using *temporary.txt* file, this file will be locked(LOCK mode), and no other thread will be able to access or modify it until T1 returns.

- When we declare a method synchronized, Java creates a “monitor” and hands it over to the thread that calls the method first time.
- As long as the thread holds the monitor, no other thread can enter the synchronized section of code.
- A monitor is like a key and the thread that holds the key can only open the lock.
- Whenever a thread has completed its work of using synchronized method, it will hand over the monitor to the next thread that is ready to use the same resource.

```
class Table  {
void printTable(int n)
{
   for(int i=1;i<=5;i++)
   {
     System.out.println(n*i);
     try{
      Thread.sleep(400);
     }catch(Exception e)
        {
         System.out.println(e);
       }
   }
 }
```

```
class MyThread1 extends Thread
{
Table t;
MyThread1(Table t){
this.t=t;
}
public void run(){
t.printTable(5);
}
}
class MyThread2 extends Thread{
Table t;
MyThread2(Table t){
this.t=t;
}
public void run()
{
t.printTable(100);
}
}

class TestSynchronization1
{
public static void main(String args[])
{
Table obj = new Table();
MyThread1 t1=new MyThread1(obj);
MyThread2 t2=new MyThread2(obj);
t1.start();
t2.start();
}
}
```

Output:
5
100
10
200
15
300
20
400
25
500

```
class Table{
 synchronized void printTable(int n)
{
   for(int i=1;i<=5;i++)
   {
     System.out.println(n*i);
     try{
      Thread.sleep(400);
     }catch(Exception e){System.out.println(e);}
   }
  }
}
```

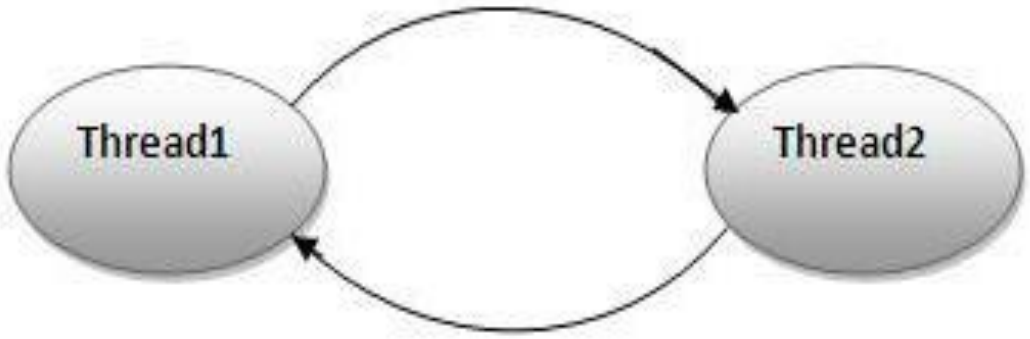
```
class MyThread1 extends Thread
{
Table t;
MyThread1(Table t){
this.t=t;
}
public void run(){
t.printTable(5);
}
}
class MyThread2 extends Thread{
Table t;
MyThread2(Table t){
this.t=t;
}
public void run()
{
t.printTable(100);
}
}

class TestSynchronization1
{
public static void main(String args[])
{
Table obj = new Table();
MyThread1 t1=new MyThread1(obj);
MyThread2 t2=new MyThread2(obj);
t1.start();
t2.start();
}
}
```

Output:
5
10
15
20
25
100
200
300
400
500

Deadlock

- Deadlock can occur in a situation when a thread is waiting for an object lock, that is acquired by another thread and second thread is waiting for an object lock that is acquired by first thread. Since, both threads are waiting for each other to release the lock, the condition is called deadlock.



Inter-thread communication

- **Inter-thread communication** or **Co-operation** is all about allowing synchronized threads to communicate with each other.
- Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed. It is implemented by following methods
  - wait()
  - notify()
  - notifyAll()

1) wait() method

- Causes current thread to release the lock and wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.

2) notify() method

- Wakes up a single thread that is waiting on this object's monitor. If any threads are waiting on this object, one of them is chosen to be awakened.

3) notifyAll() method

- Wakes up all threads that are waiting on this object's monitor.

Thread Exceptions

- The call to **sleep()** method is in a try block and followed by a catch block. Because the **sleep()** method throws an **InterruptedException**, which should be caught otherwise program will not compile.
- Java runtime system will throw an **IllegalThreadStateException** whenever we attempt to invoke a method that a thread cannot handle in the given state.
- E.g. a sleeping thread cannot deal with the **resume()** method because a sleeping thread cannot receive any instructions. Similarly we cannot use **suspend()** method on a blocked thread.